实验报告：基于RNN/LSTM的诗歌生成

1 模型原理

1.1 RNN（循环神经网络）

RNN（Recurrent Neural Network）是一种专门用于处理序列数据的神经网络。其核心思想是引入“循环连接”，使得网络在处理当前时刻的输入时，能够保留前一时刻的隐藏状态信息，从而捕捉序列中的时间依赖关系。

优点：能够处理任意长度的序列，参数共享。

缺点：存在梯度消失或梯度爆炸问题，难以捕捉长距离依赖。

1.2 LSTM（长短期记忆网络）

LSTM（Long Short-Term Memory）是RNN的一种改进变体，通过引入“门控机制”（输入门、遗忘门、输出门）和“细胞状态”，有效解决了传统RNN的梯度消失问题，能够更好地捕捉长距离依赖。

- **遗忘门**：决定丢弃哪些历史信息。
- **输入门**：决定将哪些新信息存入细胞状态。
- **输出门**：决定当前时刻的输出。

在本实验中，模型使用了两层LSTM结构，每层隐藏单元数为128。

1.3 GRU（门控循环单元）

GRU（Gated Recurrent Unit）是LSTM的简化版本，将遗忘门和输入门合并为“更新门”，并将细胞状态与隐藏状态合并，结构更简单，参数量更少，训练速度更快，在许多任务中表现与LSTM相当。

区别：

- LSTM：三个门 + 两个状态（细胞状态、隐藏状态）
- GRU：两个门 + 一个状态（隐藏状态）

2 诗歌生成过程详解

本实验基于字符级RNN模型，通过训练唐诗数据集，实现给定起始字后的古诗自动生成。整体流程如下：

2.1 数据处理

- **数据来源**：`poems.txt`（每行格式：`标题:内容`）

- **预处理**：
  a. 去除标题，提取诗歌内容。
  b. 去除空格、标点符号（`，`、`。`）及非法字符（如`_`、`（`、`《`等）。
  c. 过滤长度过短（<5）或过长（>80）的诗歌。
  d. 在每首诗前后添加起止符：`G`（start_token）和`E`（end_token）。
  e. 按诗歌长度排序，构建词频字典，将每个字映射为整数索引。
  f. 将所有诗歌转换为整数向量序列。

2.2 模型结构

模型定义在`rnn.py`中，主要包括：

- **词嵌入层**：将输入的整数索引映射为100维的稠密向量。
- **LSTM层**：两层LSTM，隐藏单元数为128，输入为词嵌入向量，输出为每个时间步的隐藏状态。
- **全连接层 + LogSoftmax**：将LSTM输出映射为词表大小的概率分布。

代码块

```
class RNN_model(nn.Module):
    def __init__(self, batch_sz ,vocab_len ,word_embedding,embedding_dim, lstm_hidden_dim):
        super(RNN_model,self).__init__()

        self.word_embedding_lookup = word_embedding
        self.batch_size = batch_sz
        self.vocab_length = vocab_len
        self.word_embedding_dim = embedding_dim
        self.lstm_dim = lstm_hidden_dim
        ########################################
        # here you need to define the "self.rnn_lstm"  the input size is "embedding_dim" and the output size is "lstm_hidden_dim"
        # the lstm should have two layers, and the  input and output tensors are provided as (batch, seq, feature)
        self.rnn_lstm = nn.LSTM(embedding_dim, lstm_hidden_dim, num_layers=2, batch_first=True)
        ########################################
        self.fc = nn.Linear(lstm_hidden_dim, vocab_len )
        self.apply(weights_init) # call the weights initial function.

        self.softmax = nn.LogSoftmax() # the activation function.
        # self.tanh = nn.Tanh()
    def forward(self,sentence,is_test = False):
        batch_input = self.word_embedding_lookup(sentence).view(1,-1,self.word_embedding_dim)
```

```
        # print(batch_input.size()) # print the size of the input
        ##############################################
        # here you need to put the "batch_input"  input the self.lstm which is defined before.
        # the hidden output should be named as output, the initial hidden state and cell state set to zero.
        output, (h_n, c_n) = self.rnn_lstm(batch_input)
        ##############################################
        out = output.contiguous().view(-1,self.lstm_dim)

        out =  F.relu(self.fc(out))

        out = self.softmax(out)

        if is_test:
            prediction = out[ -1, : ].view(1,-1)
            output = prediction
        else:
           output = out
        # print(out)
        return output
```

2.3 训练过程

- **优化器**：RMSprop（学习率0.01）
- **损失函数**：NLLLoss（负对数似然损失）
- **训练策略**：
  - 按批次训练（batch_size=100），每个批次内逐条诗歌计算损失并累加。
  - 梯度裁剪（clip_grad_norm_ = 1）防止梯度爆炸。
  - 每20个batch保存一次模型。

训练时，模型根据当前字预测下一个字，输入为 `[w1, w2, ..., wn]`，目标为 `[w2, w3, ..., wn, wn]`。

截图如下：

```
******************************
epoch  29 batch number 344 loss is:  5.999762535095215
prediction [10, 7, 91, 97, 117, 0, 10, 117, 91, 6, 127, 1, 10, 17, 6, 62, 61, 0, 7, 21, 16, 54, 56, 1, 7, 32, 240, 4, 61, 0, 11, 85, 255, 15, 46
, 1, 8, 48, 6, 9, 61, 0, 5, 21, 8, 54, 240, 1, 15, 50, 74, 267, 45, 0, 17, 18, 19, 6, 131, 1, 3, 17, 9, 69, 74, 0, 7, 24, 4, 82, 20, 1, 3, 3]
b_y        [196, 330, 69, 44, 70, 0, 581, 451, 18, 714, 457, 1, 8, 169, 5, 24, 73, 0, 6, 40, 100, 54, 626, 1, 185, 11, 32, 1148, 1148, 0, 249, 1
6, 63, 236, 236, 1, 118, 48, 168, 450, 378, 0, 6, 21, 101, 289, 615, 1, 22, 50, 259, 232, 252, 0, 33, 364, 117, 93, 594, 1, 233, 161, 88, 201, 9
43, 0, 86, 30, 43, 39, 632, 1, 3, 3]
******************************
epoch  29 batch number 345 loss is:  6.19264554977417
prediction [10, 18, 8, 51, 940, 0, 8, 8, 41, 30, 7, 1, 10, 6, 117, 182, 63, 0, 182, 48, 5, 5, 32, 1, 4, 18, 5, 168, 16, 0, 6, 32, 191, 21, 137,
1, 82, 16, 47, 7, 59, 0, 25, 116, 16, 17, 17, 1, 15, 172, 4, 190, 14, 0, 4, 480, 10, 62, 59, 1, 3, 96, 9, 282, 29, 0, 42, 0, 4, 79, 4, 96, 10, 2
99, 5, 1, 3, 3]
b_y        [457, 152, 475, 147, 70, 0, 223, 8, 37, 313, 1117, 1, 302, 457, 231, 652, 1824, 0, 222, 704, 70, 147, 5, 1, 457, 48, 84, 88, 16, 0, 2
01, 72, 1766, 372, 1209, 1, 88, 16, 112, 594, 93, 0, 12, 57, 56, 23, 79, 1, 149, 278, 80, 309, 2542, 0, 28, 175, 254, 217, 1113, 1, 271, 488, 45
0, 135, 43, 15, 42, 0, 12, 65, 564, 475, 540, 157, 29, 1, 3, 3]
******************************
epoch  29 batch number 346 loss is:  6.262526035308838
prediction [10, 23, 43, 12, 18, 0, 65, 0, 10, 59, 28, 100, 10, 58, 21, 1, 10, 11, 4, 34, 261, 11, 6, 0, 10, 186, 28, 268, 16, 97, 107, 1, 82, 16
, 35, 6, 16, 41, 41, 0, 10, 11, 28, 73, 69, 26, 32, 1, 4, 5, 4, 163, 32, 46, 5, 0, 327, 6, 243, 25, 268, 25, 132, 1, 3, 163, 4, 11, 9, 108, 132,
 0, 4, 131, 15, 37, 10, 6, 1, 1, 3, 3]
b_y        [188, 682, 804, 242, 143, 242, 70, 0, 10, 242, 297, 41, 10, 242, 100, 1, 342, 71, 191, 6, 240, 60, 154, 0, 555, 85, 1335, 1335, 69, 9
7, 107, 1, 139, 284, 325, 196, 4, 149, 95, 0, 36, 11, 99, 73, 144, 37, 381, 1, 38, 5, 112, 163, 1572, 241, 867, 0, 182, 7, 1076, 1076, 243, 134,
 434, 1, 441, 70, 296, 457, 869, 108, 20, 0, 729, 131, 907, 117, 66, 6, 539, 1, 3, 3]
******************************
epoch  29 batch number 347 loss is:  6.105762004852295
```

2.4 生成过程

- 给定起始字（如“日”），将其转换为索引。
- 将已生成的诗句输入模型，得到下一个字的概率分布。
- 取概率最大的字作为预测结果，加入诗句。
- 重复上述过程，直到生成结束符 `E` 或超过最大长度（30字）。
- 最后对生成的诗句进行格式化输出，按“。”分行显示。

3 实验总结与反思

3.1 实验结果

使用约30轮训练后，模型能够生成具有一定通顺度和诗意的句子。（由于pretty_print_poem会删去一些不够长的句子，首字符可能无法显示，因此我使用如下方式打印结果）：

代码块

```
print("==================原始模型输出==================")
print(gen_poem("日"))
print(gen_poem("红"))
print(gen_poem("山"))
print(gen_poem("夜"))
print(gen_poem("湖"))
print(gen_poem("君"))

print("\n==================美化输出后的古诗==================")
pretty_print_poem(gen_poem("日"))
pretty_print_poem(gen_poem("红"))
pretty_print_poem(gen_poem("山"))
pretty_print_poem(gen_poem("夜"))
pretty_print_poem(gen_poem("湖"))
pretty_print_poem(gen_poem("君"))
```

```
==================原始模型输出==================
error
inital  linear weight
/home/ljx/anaconda3/envs/m2track/lib/python3.10/site-packages/torch/nn/m
ftmax has been deprecated. Change the call to include dim=X as an argume
  return self._call_impl(*args, **kwargs)
日萧萧。玉座临江水，金门不及功。人间不得在，人似有谁生。E
error
inital  linear weight
红尘。云月临江口，风光入远山。云山连万树，风起水中风。何处无人
error
inital  linear weight
山上一万峰，千门万树发秋风。云山不见日，云树日无风。何处无人起
error
inital  linear weight
夜寒风，香风起翠云。风吹玉树远，月上玉人开。一望寒峰起，千门万
error
inital  linear weight
湖南日日，风起云中树。云生无远山，风起天上楼。何处无人事，何人
error
inital  linear weight
君子心相似。今日见时事，今人无事人。今朝不见远，今日不如人。今
```

```
==================美化输出后的古诗==================
error
inital  linear weight
玉座临江水，金门不及功。
人间不得在，人似有谁生。
error
inital  linear weight
云月临江口，风光入远山。
云山连万树，风起水中风。
error
inital  linear weight
山上一万峰，千门万树发秋风。
云山不见日，云树日无风。
error
inital  linear weight
风吹玉树远，月上玉人开。
error
inital  linear weight
云生无远山，风起天上楼。
error
inital  linear weight
今日见时事，今人无事人。
今朝不见远，今日不如人。
```

可以看到模型具备了一定的生成古诗的能力。

3.2 实验收获

1. **深入理解了RNN/LSTM在序列生成中的应用**：通过实现字符级语言模型，掌握了序列数据的处理方式、时间步的传播机制。
2. **掌握了文本生成的基本流程**：从数据清洗、词表构建、批次生成到模型训练与推理。
3. **了解了超参数的影响**：如隐藏层维度、学习率、批次大小对收敛速度和生成质量的影响。

3.3 存在的问题与改进方向

- **重复生成**：部分生成内容出现字词重复，可通过引入温度采样（temperature sampling）或top-k采样增加多样性。
- **上下文连贯性不足**：模型仅基于前几个字预测，缺乏整体语义规划，可引入注意力机制或Transformer结构。
- **数据量有限**：当前数据集规模较小（约4万首），使用更大规模数据集（如全唐诗）可提升效果。
- **训练效率**：当前逐条累加损失的方式较慢，可优化为批量并行计算。